DATA 266 Homework 4.5

DATA 266 — Homework 4.5

Sneha Singh
SID4 = 0670 | SEED = 670 | SLICE = 670 | HP_ID = 4 | CLS_A = 0 | CLS_B = 5

Notebook: `hw4.5.ipynb`. GPU: NVIDIA L4 on Colab Pro. Part C skipped (optional). One GPU.

Same standing parameters as HW1. HP_ID is reported only. Every number below is from the executed Colab notebook and from `RUN_LOG.txt` / `METRICS.md`.

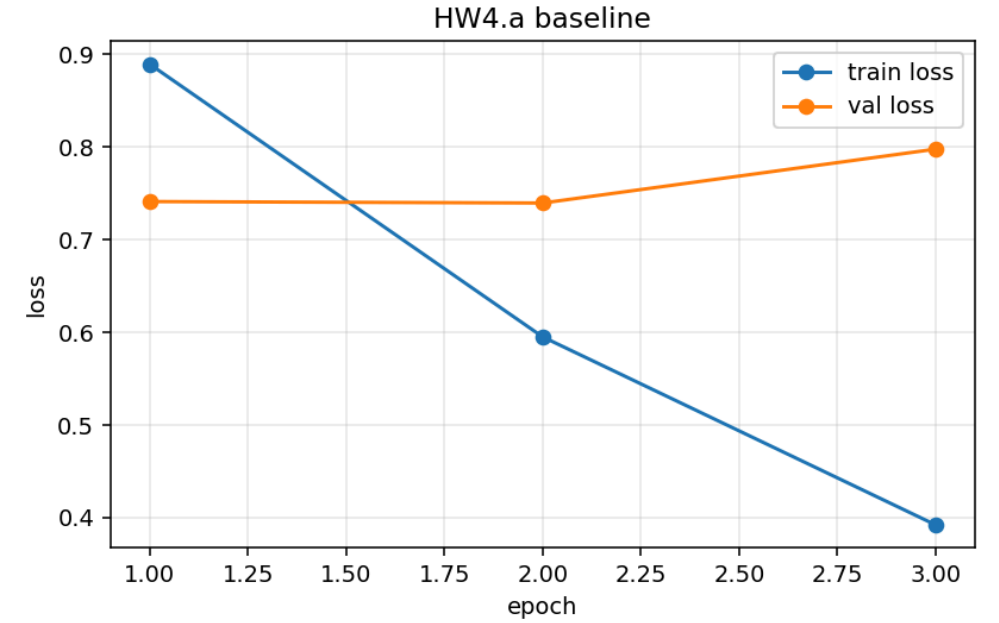
Setup

Item	Value
Data	`cardiffnlp/tweet_eval` sentiment, train split only
Rows	670–5669 inclusive (5,000 consecutive)
Split	3,500 / 750 / 750 with SEED 670
Labels in slice	negative 773, neutral 2290, positive 1937
Model	`bert-base-uncased`, BertTokenizer, max length 128
Baseline train	batch 16, AdamW, lr 2e-5, weight decay 0.01, 3 epochs
Serving model	`facebook/opt-125m`, 32 new tokens, greedy
Device	NVIDIA L4 (Colab Pro)

HW4.a — BERT sentiment baseline

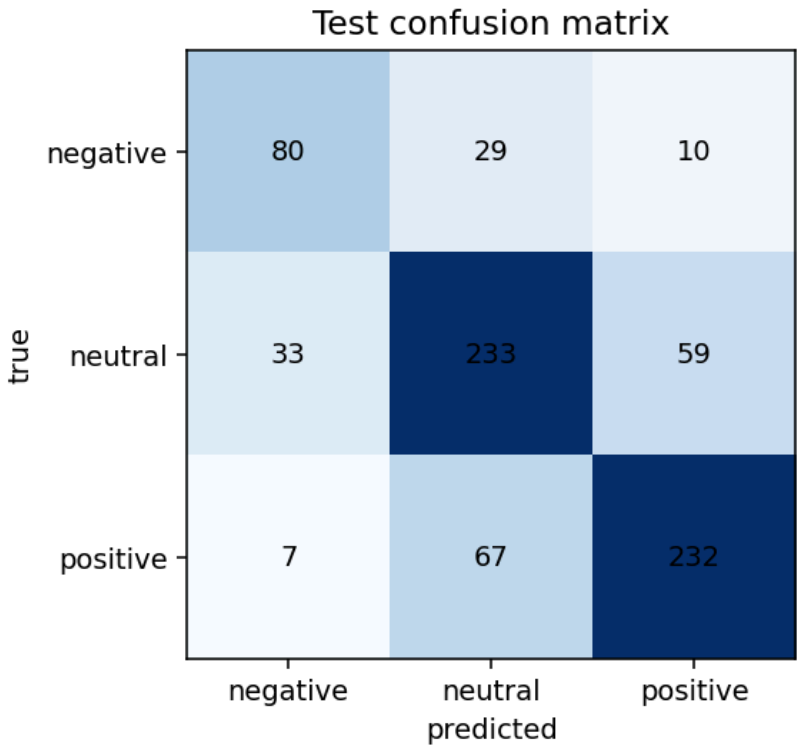
Validation loss was lowest at **epoch 2** (0.7395). Epoch 3 went to 0.7976, so I kept the epoch-2 weights. Training past that point cost about **38.3 s** (the third epoch).

Epoch	Train loss	Val loss	Val accuracy	Seconds
1	0.8892	0.7410	0.6640	37.0
2	0.5950	0.7395	0.6773	37.8
3	0.3922	0.7976	0.6813	38.3



Baseline loss

Peak allocated memory: **2.48 GB**. Wall-clock: **113.1 s**. Test accuracy at best val epoch: **0.7267**.



Confusion matrix

Class	Precision	Recall	Support
negative	0.6667	0.6723	119
neutral	0.7082	0.7169	325
positive	0.7708	0.7582	306

Worst class: **negative**. Five misclassified negatives (model predicted neutral):

1. *“Sophos is the Lamest Virus Ever…”* — insult is there, but the rest reads like a headline.
2. *“Because she’s Caitlyn Jenner…”* — sarcasm / implication, no clear negative word.
3. *“Tonight may be the night when I finally expose myself to the Gilmore Girls hysteria”* — ironic tone.
4. *“Monday was Gucci but then it went downhill”* — flips mid-sentence.
5. *“Hahaha 2nd place in Miss USA…”* — sarcasm wrapped around a joke.

Reload on three held-out sentences: positive / negative / neutral.

Table HW4.1 — Serving comparison

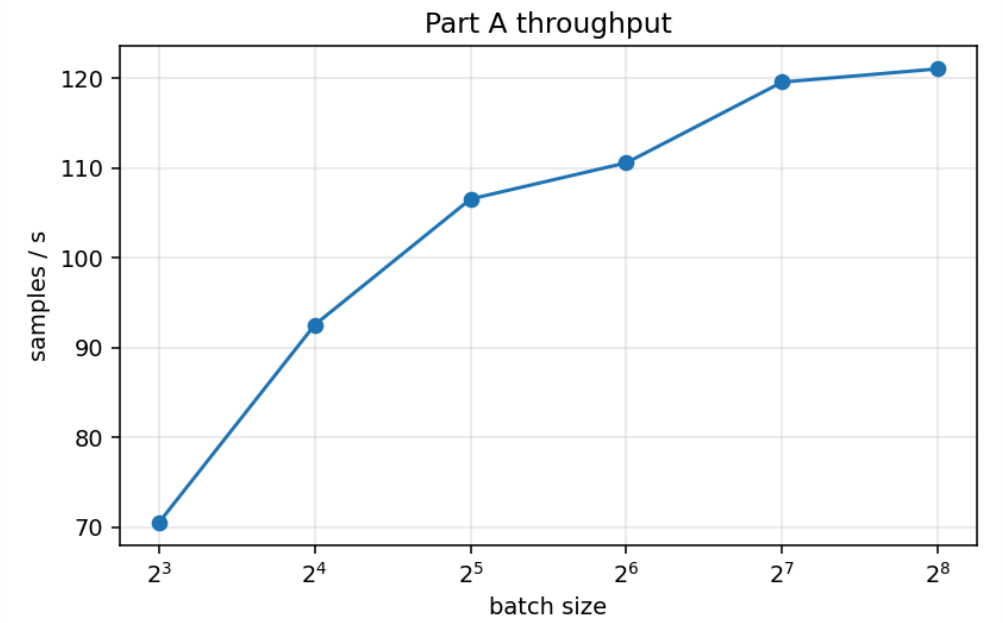
Same 32 prompts, `facebook/opt-125m`, float16, 32 new tokens.

Configuration	Tokens/sec	p50 latency (ms)	p95 latency (ms)
HF pipeline, batch 1	115.7	269.3	280.8
vLLM, batch 1	97.8	319.1	328.4
vLLM, batch 8	627.8	349.2	354.9
vLLM, batch 32	2273.5	383.7	386.5

At **batch 8**, p50 is worse than the HF batch-1 baseline (349 ms vs 269 ms), but tokens/s jumps from 98 to 628, so throughput is still improving while per-request latency is worse. Batch 32 continues that pattern. For an interactive app I care about **p50**. Tokens/s matters when many requests share the GPU.

Part A — batch scaling

Batch	Samples/s	Peak GB	Seconds / epoch
8	70.5	2.09	49.7
16	92.5	2.49	37.8
32	106.5	3.76	32.9
64	110.6	6.15	31.7
128	119.6	11.00	29.3
256	121.0	20.65	28.9
512	OOM	—	—



Part A throughput

Largest batch: **256**. Throughput within 5% of peak from batch **128**. One GPU only (TA: one is enough), so no second-card ratio. Batch ratio would still not equal VRAM ratio because weights, Adam state, and CUDA context are fixed cost.

Part B — memory techniques

Table HW4.5.1 — Report this table once per card (NVIDIA L4):

Technique	Max batch	Throughput (samp/s)	Peak memory (GB)	Eval accuracy
Baseline (HW4 settings)	256	92.8	2.48	0.7267
Mixed precision	256	189.3	2.26	0.7040
Gradient checkpointing	1024	70.7	2.09	0.7267
Gradient accumulation	128 (physical 8)	99.5	2.32	0.6520
8-bit optimiser	256	99.3	1.90	0.7187
Best combination	256	222.0	1.64	0.7160

Gradient checkpointing: 92.8 → 70.7 samp/s (**–23.8%**). Accumulation same-step wall-clock: **816.9 s** vs baseline **113.1 s**.

Rank	Technique	Saved GB	Δ samp/s	Saved GB / samp/s lost
1	8-bit optimiser	0.58	+6.5	free — apply first
2	Mixed precision	0.22	+96.5	free
3	Gradient accumulation	0.16	+6.7	free
4	Gradient checkpointing	0.39	−22.1	0.018

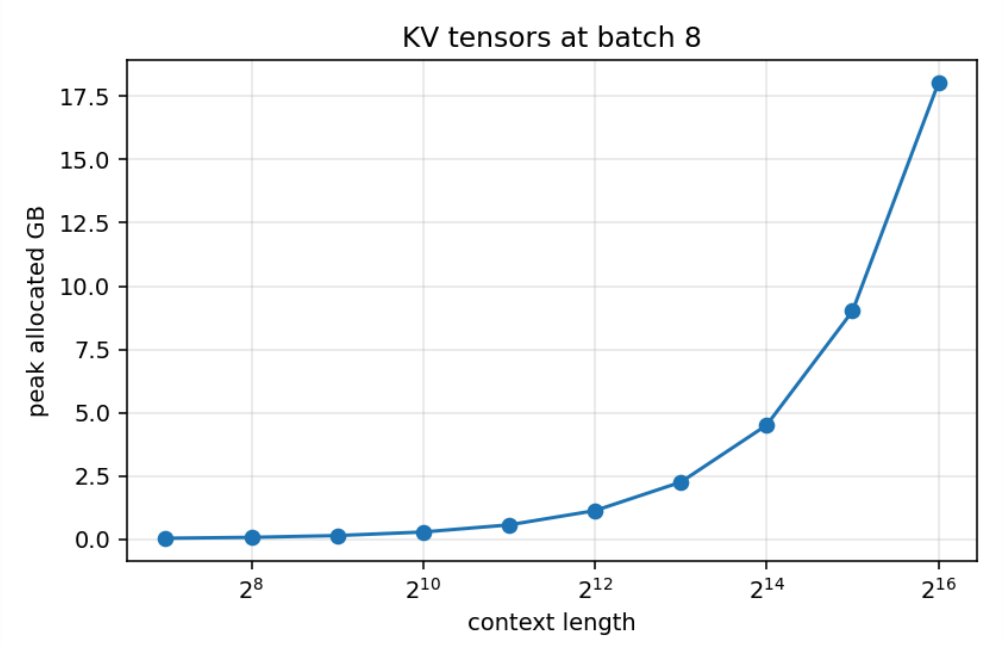
Apply first: 8-bit AdamW. Not worth on a 32 GB card here: gradient checkpointing. Best combination: AMP + 8-bit AdamW.

Part C

Part C skipped (optional).

Part D — serving revisited and KV cache

Part D vLLM re-run on the same L4: batch 1 / 8 / 32 → 97.9 / 622.9 / 2354.5 tok/s. Nearly identical to Table HW4.1 (same hardware and float16).



KV cache

Context	Peak GB
128	0.05
256	0.09
512	0.16
1024	0.30
2048	0.58
4096	1.14
8192	2.27
16384	4.52
32768	9.02
65536	18.02

Longest context at batch 8 that fit: **65536** (no allocation OOM on this ladder).

Part E — Analysis and summary

GPU-hours for one full fine-tune

Using the HW4.a baseline wall-clock on this L4:

GPU-hours = 113.1 s / 3600 = **0.0314**

That is training time only, not Colab seat time.

Single configuration I would use

Best combination (AMP + 8-bit AdamW), from Table HW4.5.1: peak memory **1.64 GB**, throughput **222.0** samples/s, wall-clock **47.3 s**, eval accuracy **0.7160**. It is the fastest row that stays within about 1.5 points of the baseline accuracy (0.7267), and it does not need checkpointing on a card that already fits batch 256.

HW4.5 summary (also in `METRICS.md`)

This is the HW4.5 summary. It is **not** an HW5 baseline.

Table HW4.5.1 — Report this table once per card (same L4):

Technique	Max batch	Throughput (samp/s)	Peak memory (GB)	Eval accuracy
Baseline (HW4 settings)	256	92.8	2.48	0.7267
Mixed precision	256	189.3	2.26	0.7040
Gradient checkpointing	1024	70.7	2.09	0.7267
Gradient accumulation	128 (physical 8)	99.5	2.32	0.6520
8-bit optimiser	256	99.3	1.90	0.7187
Best combination	256	222.0	1.64	0.7160

Conclusion

On the L4, BERT-base at batch 16 is not memory-bound (2.48 GB). Throughput stops improving much by batch 128; OOM at 512. Mixed precision and 8-bit AdamW are the useful changes. For serving, larger vLLM batches raise tokens/s while p50 gets worse than HF batch 1, so an interactive demo should watch latency.